COMPUTER VISION

Subject Code: 21CSE454T

ASSIGNMENT - UNIT 4

Image Segmentation

Task:

Implement region segmentation using Otsu's Adaptive Thresholding and Watershed Segmentation to cleanly isolate distinct foreground objects from highly varied background lighting conditions.

SUBMITTED BY

Name	**:**	Srevarshan
Register Number	**:**	RA2311026050039
Year	**:**	4th Year
Department	**:**	CSE - AIML
Section	**:**	A
Subject	**:**	Computer Vision
Subject Code	**:**	21CSE454T

1. AIM

To separate the coins from the background in an image with uneven lighting using Otsu's adaptive thresholding, and to separate the touching coins using Watershed segmentation.

2. ALGORITHM / PROCEDURE

A. Otsu's Thresholding

Step 1: Read the coins image and add uneven lighting and a shadow to it.

Step 2: Smooth the image with a Gaussian filter.

Step 3: Global Otsu: find the one threshold that best separates the histogram into two classes (maximum between-class variance).

Step 4: Adaptive Otsu: find a separate Otsu threshold for a small circular region around every pixel.

Step 5: Clean the binary mask using morphological opening.

B. Watershed Segmentation

Step 1: Find the sure background by dilating the mask.

Step 2: Find the distance transform of the mask. The peaks are the centres of the coins and are used as markers.

Step 3: Mark the region between the background and the markers as unknown.

Step 4: Apply the watershed algorithm. Boundaries are drawn where the regions from two markers meet.

Step 5: Count the coins and measure their areas.

3. PYTHON CODE

File name: unit4_segmentation.py

```
"""
Unit 4 - Image Segmentation
Otsu thresholding (global vs. locally-adaptive) followed by marker-controlled
Watershed segmentation to isolate touching coins under strongly uneven
background lighting.
"""
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.filters import rank
from skimage.morphology import disk, remove_small_holes, remove_small_objects
from skimage.feature import peak_local_max

HERE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(HERE, "outputs")
os.makedirs(OUT, exist_ok=True)

# ---------------- 1. Input with highly varied lighting ----------------
src = cv2.imread(os.path.join(HERE, "water_coins.jpg"), cv2.IMREAD_GRAYSCALE)   # 24 touching
    coins
src = cv2.resize(src, None, fx=2, fy=2, interpolation=cv2.INTER_CUBIC)
h, w = src.shape
yy, xx = np.mgrid[0:h, 0:w]
light = 1.35 - 1.05 * (0.6 * xx / w + 0.4 * yy / h)                    # bright top-left ->
    dark bottom-right
light -= 0.30 * np.exp(-((xx - 0.2 * w) ** 2 + (yy - 0.75 * h) ** 2) / (2 * 90 ** 2))   # cast
     shadow
img = np.clip(src * light, 0, 255).astype(np.uint8)
cv2.imwrite(os.path.join(OUT, "input.png"), img)
smooth = cv2.GaussianBlur(img, (5, 5), 0)


# ---------------- 2. Otsu's method (from scratch) ----------------
def otsu_threshold(gray):
    hist = np.bincount(gray.ravel(), minlength=256).astype(np.float64)
    p = hist / hist.sum()
    omega = np.cumsum(p)                          # class-0 probability
    mu = np.cumsum(p * np.arange(256))            # class-0 cumulative mean
    with np.errstate(divide="ignore", invalid="ignore"):
        sigma_b2 = (mu[-1] * omega - mu) ** 2 / (omega * (1 - omega))   # between-class
            variance
    sigma_b2 = np.nan_to_num(sigma_b2)
    return int(np.argmax(sigma_b2)), sigma_b2


# (a) Global Otsu - one threshold for the whole image (coins are darker -> inverse)
t_glob, sb = otsu_threshold(smooth)
cv_t, _ = cv2.threshold(smooth, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
glob = np.where(smooth <= t_glob, 255, 0).astype(np.uint8)

# (b) Adaptive Otsu - Otsu threshold computed in a sliding disk around every pixel.
#     Flat (low-contrast) windows contain only background and are rejected.
R = 45
T_local = rank.otsu(smooth, disk(R))
contrast = rank.maximum(smooth, disk(R)).astype(int) - rank.minimum(smooth, disk(R))
adapt = ((smooth <= T_local) & (contrast > 25)).astype(np.uint8) * 255

print(f"Image size                  : {w} x {h}")
print(f"Global Otsu threshold       : {t_glob}  (cv2 THRESH_OTSU = {int(cv_t)})")
print(f"Adaptive Otsu window        : disk radius {R}px, local T range
    {T_local.min()}-{T_local.max()}")


def cleanup(m):
```

```
    k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    m = cv2.morphologyEx(m, cv2.MORPH_OPEN, k, iterations=2)
    m = remove_small_holes(m > 0, max_size=100)          # fill specks inside coins only
    return remove_small_objects(m, max_size=4000).astype(np.uint8) * 255   # drop dirt / noise


glob_c, mask = cleanup(glob), cleanup(adapt)

# ---------------- 3. Marker-controlled Watershed ----------------
k3 = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
sure_bg = cv2.dilate(mask, k3, iterations=3)                       # certainly background
    outside this
dist = cv2.distanceTransform(mask, cv2.DIST_L2, 5)                 # distance to nearest
    background
peaks = peak_local_max(dist, min_distance=20, threshold_rel=0.5, labels=mask > 0,
    exclude_border=False)
markers = np.zeros(mask.shape, np.int32)
for i, (y, x) in enumerate(peaks, start=2):
    cv2.circle(markers, (int(x), int(y)), 6, i, -1)                # one seed per coin
unknown = cv2.subtract(sure_bg, (markers > 0).astype(np.uint8) * 255)
markers[sure_bg == 0] = 1                                          # background seed
color = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
ws = cv2.watershed(color, markers.copy())                          # flood from seeds; -1 =
    ridge

# ---------------- 4. Evaluation ----------------
n_glob = cv2.connectedComponents(glob_c)[0] - 1
n_adapt = cv2.connectedComponents(mask)[0] - 1
labels = [l for l in np.unique(ws) if l > 1]
areas = np.array([(ws == l).sum() for l in labels])
print(f"\nGround-truth coins                    : 24")
print(f"Regions after global Otsu             : {n_glob}")
print(f"Regions after adaptive Otsu           : {n_adapt}   (touching coins still merged)")
print(f"Seed markers from distance transform  : {len(peaks)}")
print(f"Objects after Watershed               : {len(labels)}")
print(f"Coin area (px): min={areas.min()}  max={areas.max()}  mean={areas.mean():.0f}
    std={areas.std():.0f}")

# Visual output
rng = np.random.default_rng(3)
lut = rng.integers(70, 255, (ws.max() + 1, 3)).astype(np.uint8)
lut[1] = 0
seg = lut[np.maximum(ws, 0)]
seg[ws == -1] = 255
overlay = color.copy()
overlay[ws > 1] = (0.45 * overlay[ws > 1] + 0.55 * lut[ws[ws > 1]]).astype(np.uint8)
ridge = (ws == -1).astype(np.uint8)
ridge[[0, -1], :] = 0; ridge[:, [0, -1]] = 0                       # ignore image frame
overlay[cv2.dilate(ridge, k3) > 0] = (0, 0, 255)
cents = {l: np.argwhere(ws == l).mean(0) for l in labels}
for n, l in enumerate(sorted(labels, key=lambda l: (cents[l][0] // 60, cents[l][1])),
    start=1):
    ys, xs = np.nonzero(ws == l)
    cv2.putText(overlay, str(n), (int(xs.mean()) - 9, int(ys.mean()) + 7),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2, cv2.LINE_AA)
cv2.imwrite(os.path.join(OUT, "watershed_result.png"), overlay)

# ---------------- 5. Figures ----------------
fig, ax = plt.subplots(2, 2, figsize=(11, 13.6))
for a, (t, im) in zip(ax.ravel(), [("Input (uneven lighting + shadow)", img),
                           (f"Global Otsu (T = {t_glob})", glob_c),
                           (f"Local Otsu threshold map (r = {R})", T_local),
                           ("Adaptive (local) Otsu", mask)]):
    a.imshow(im, cmap="gray"); a.set_title(t, fontsize=12); a.axis("off")
plt.tight_layout(); plt.savefig(os.path.join(OUT, "fig1_otsu.png"), dpi=110); plt.close()

fig, a = plt.subplots(figsize=(10, 3.6))
a.hist(smooth.ravel(), bins=256, range=(0, 256), color="gray")
a.axvline(t_glob, color="r", lw=2, label=f"Global Otsu T = {t_glob}")
a.set_xlabel("Intensity"); a.set_ylabel("Pixels")
```

```
a2 = a.twinx(); a2.plot(sb, "b", lw=1.2, label="Between-class variance"); a2.set_yticks([])
a.legend(loc="upper left"); a2.legend(loc="upper center")
a.set_title("Histogram - lighting spreads coin & background intensities so they overlap")
plt.tight_layout(); plt.savefig(os.path.join(OUT, "fig2_histogram.png"), dpi=110); plt.close()

fig, ax = plt.subplots(2, 2, figsize=(11, 13.6))
ax = ax.ravel()
ax[0].imshow(dist, cmap="magma"); ax[0].plot(peaks[:, 1], peaks[:, 0], "c+", ms=12, mew=2)
ax[0].set_title(f"Distance transform + {len(peaks)} seeds")
ax[1].imshow(unknown, cmap="gray"); ax[1].set_title("Unknown region (flooded)")
ax[2].imshow(cv2.cvtColor(seg, cv2.COLOR_BGR2RGB)); ax[2].set_title("Watershed labels +
    ridges")
ax[3].imshow(cv2.cvtColor(overlay, cv2.COLOR_BGR2RGB)); ax[3].set_title(f"Final: {len(labels)}
     coins isolated")
for a in ax: a.axis("off")
plt.tight_layout(); plt.savefig(os.path.join(OUT, "fig3_watershed.png"), dpi=110); plt.close()
print("\nAll figures saved in", OUT)
```

4. INPUT IMAGE



Figure 1: Input image - 24 touching coins with added uneven lighting. Source: OpenCV sample image

5. OUTPUT SCREENSHOTS

```
Terminal

$ python unit4_segmentation.py
Image size                  : 504 x 624
Global Otsu threshold       : 141  (cv2 THRESH_OTSU = 141)
Adaptive Otsu window        : disk radius 45px, local T range 0-254

Ground-truth coins                   : 24
Regions after global Otsu            : 1
Regions after adaptive Otsu          : 1   (touching coins still merged)
Seed markers from distance transform  : 24
Objects after Watershed              : 24
Coin area (px): min=6565  max=7396  mean=6972  std=189

All figures saved in Unit4_Image_Segmentation/outputs
```

Figure 2: Program output in the terminal

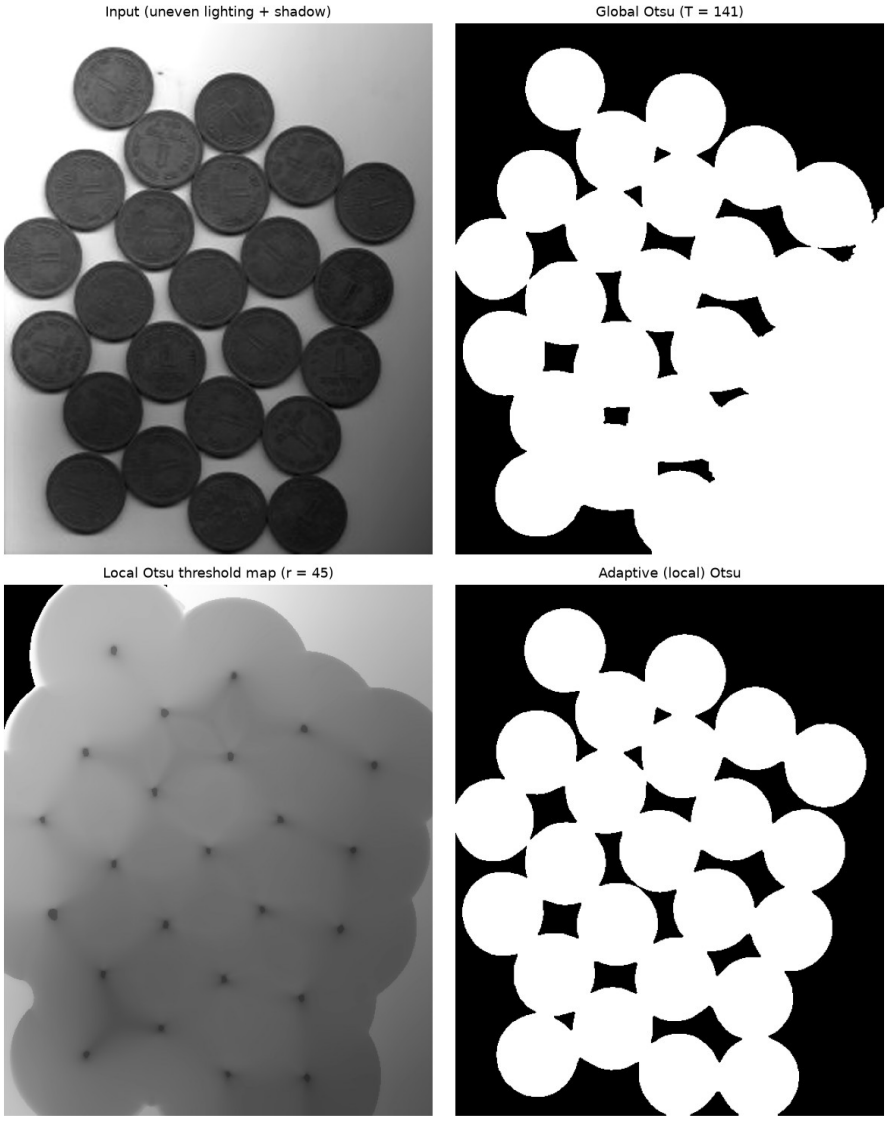


Figure 3: Global Otsu and adaptive Otsu thresholding

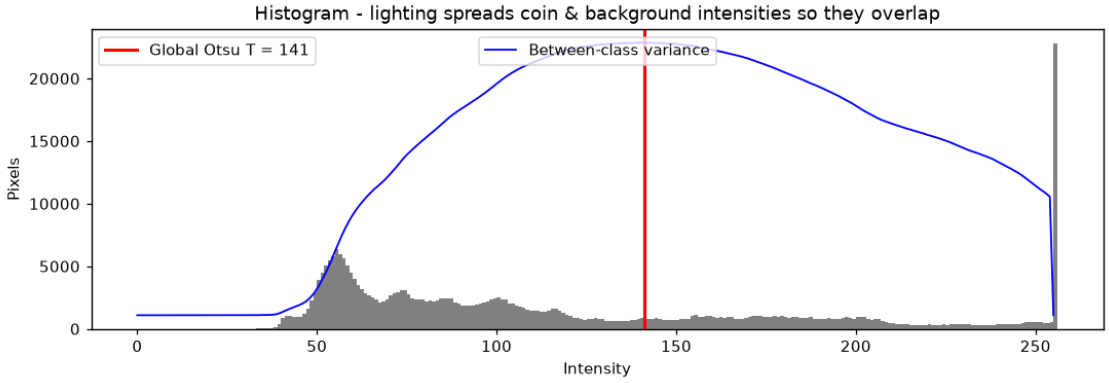


Figure 4: Histogram with the global Otsu threshold

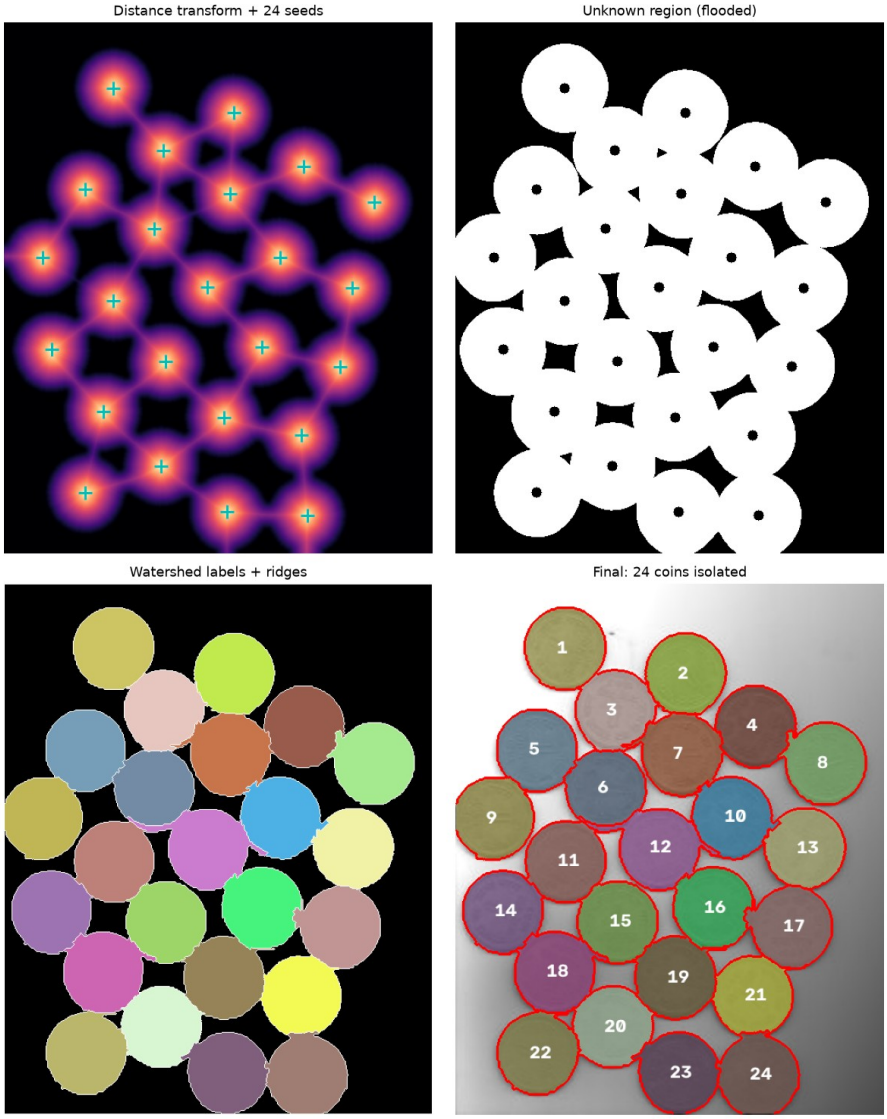


Figure 5: Steps of watershed segmentation

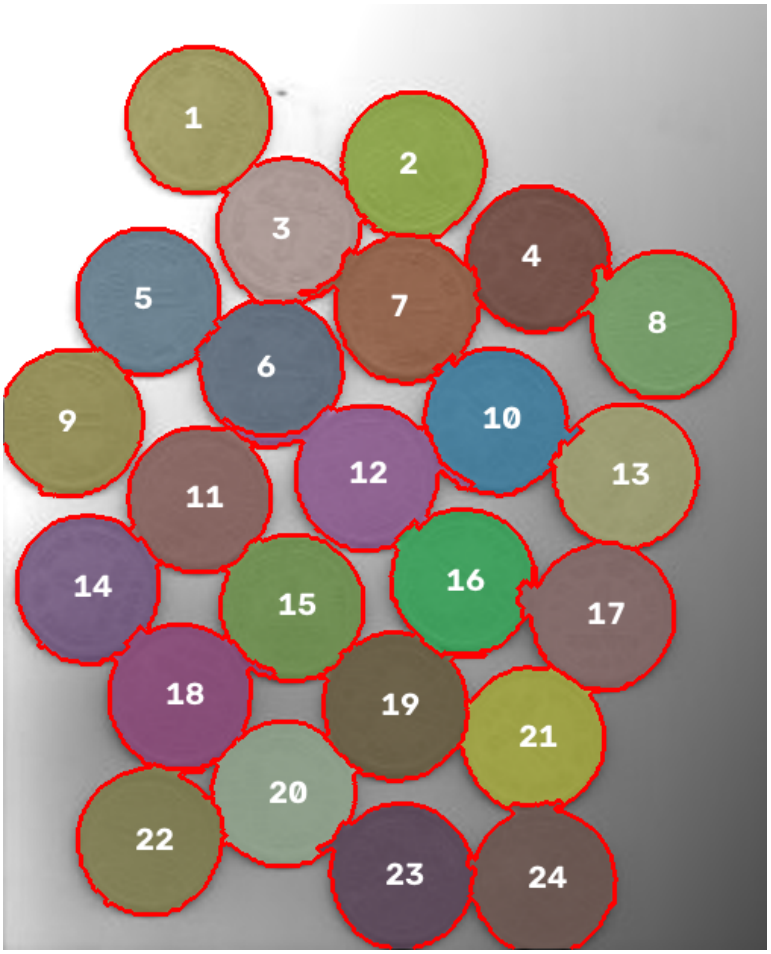


Figure 6: Final result - each coin separated and numbered

6. RESULT ANALYSIS / OBSERVATIONS

Method	Result
Global Otsu (T = 141)	Dark background in the shadow is wrongly taken as coins
Adaptive Otsu	Correct coin mask, but touching coins are joined (1 region)
Markers found	24
Watershed segmentation	24 coins separated (actual = 24)
Coin area	6565 to 7396 pixels (average 6972)

- Global Otsu does not work here because the background in the dark corner is darker than the coins in the bright corner, so one threshold cannot separate them.
- Adaptive Otsu uses a different threshold for each region, so it handles the uneven lighting and gives a correct mask.
- Thresholding alone cannot separate coins that touch each other.
- Watershed, using the distance transform peaks as markers, separated all 24 coins.
- All the coins have similar areas, which shows that the boundaries are accurate.

7. CONCLUSION

Global Otsu thresholding fails when the lighting is uneven. Adaptive Otsu thresholding solves this by using a local threshold. Watershed segmentation then separated the touching coins, and all 24 coins were correctly isolated.